

Stimmbandläsion Web Tool

Architecture Documentation

April 7, 2026

Contents

1	Introduction	3
2	System Overview	3
3	Technology Stack	4
4	Infrastructure and Deployment	4
4.1	Docker Compose Services	4
4.2	Nginx Reverse Proxy	4
4.3	Development vs. Production	5
4.4	Build Pipeline	5
5	Back-end Architecture	5
5.1	Data Model	5
5.1.1	Patient Workflow States	6
5.1.2	Enumerations	7
5.2	REST API	7
5.2.1	Authentication Endpoints	7
5.2.2	Admin Endpoints (JWT Required)	7
5.2.3	Patient-Facing Endpoints (UUID Token)	8
5.2.4	Public Endpoints	8
5.3	Serialisers	8
5.4	Permissions	9
5.5	Business Logic Services	9
5.6	Asynchronous Tasks	9
5.7	Dependencies	9
6	Front-end Architecture	10
6.1	Routing	10
6.2	API Client	10
6.3	Patient Wizard Flow	10
6.4	Audio Recording	11
6.5	Component Overview	12
6.6	Theme	12

7	Audio Upload Flow	12
8	AI Inference Integration	12
9	Data Export	13
10	Security Considerations	13
11	Summary	13

1 Introduction

The *Stimmbandläsion* web tool is a medical data-collection platform for vocal cord lesion (recurrent laryngeal nerve paresis) diagnosis. It manages patient records, collects voice audio recordings for pre-operative and post-operative phases, integrates an AI inference service for automated prediction, and provides data export functionality for clinical research.

The application follows a containerised microservice architecture orchestrated with Docker Compose, comprising a Django REST backend, a React single-page application frontend, an Nginx reverse proxy, and supporting infrastructure services (PostgreSQL, MinIO / S3, Redis).

2 System Overview

Figure 1 illustrates the high-level request flow and service topology.

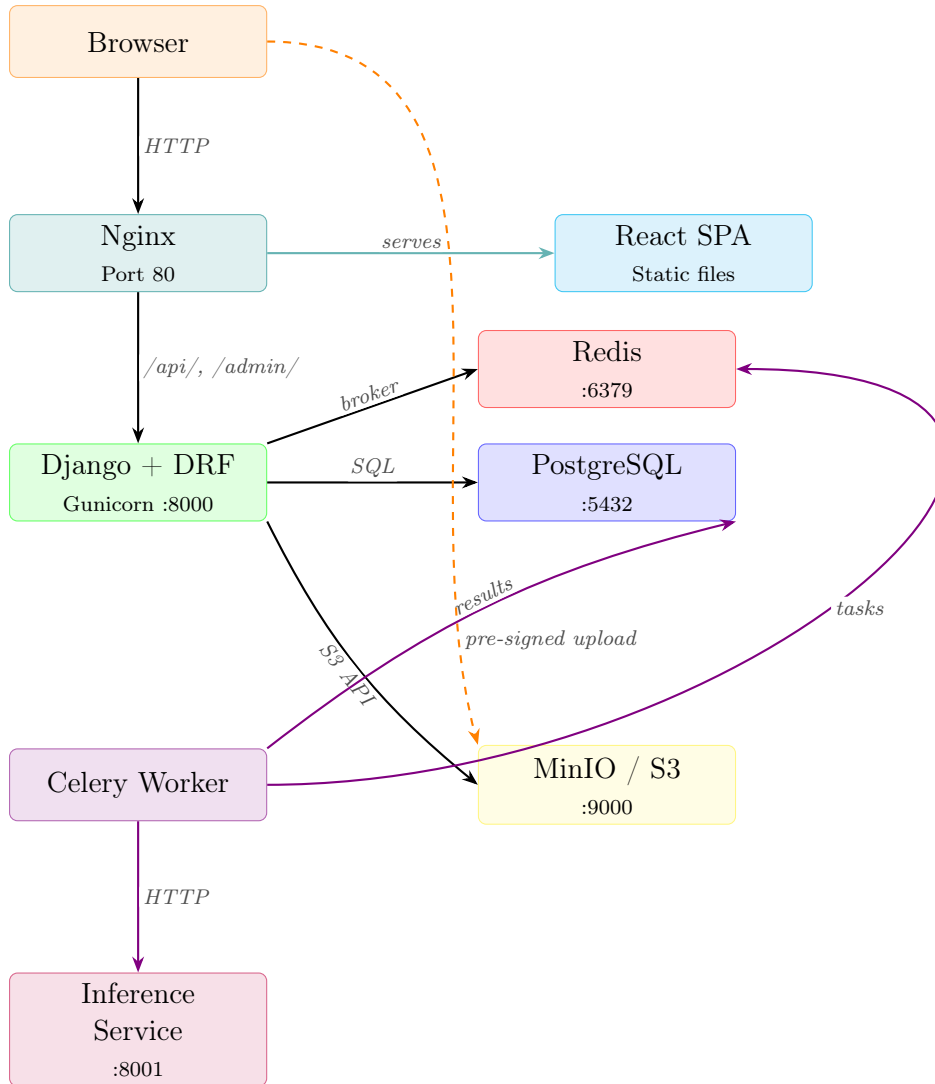


Figure 1: System overview and request flow.

3 Technology Stack

Table 1: Technology choices and rationale.

Concern	Choice	Rationale
Back-end framework	Django 5.1 + DRF	Mature ORM, built-in admin, serialisers, permissions
Front-end framework	React 19 + Vite 8	Fast builds, TypeScript, component model
UI component library	Material UI (MUI) 7	Consistent design system, rich widget set
Reverse proxy	Nginx	Serves SPA static files, proxies API
WSGI server	Gunicorn	Production-grade, multi-worker
Database	PostgreSQL 16	Relational, robust, widely supported
Object storage	MinIO (dev) / S3 (prod)	S3-compatible, self-hosted for development
Task queue	Celery + Redis	Asynchronous inference calls
Authentication	JWT (SimpleJWT)	Stateless admin auth; UUID tokens for patients
Containerisation	Docker + Compose	Reproducible, portable deployments

4 Infrastructure and Deployment

4.1 Docker Compose Services

All services are defined in `docker-compose.yml` with development overrides in `docker-compose.dev.yml`.

Table 2: Docker Compose service definitions.

Service	Image	Port(s)	Role
db	postgres:16-alpine	5432	Primary relational database
minio	minio/minio:latest	9000, 9001	S3-compatible object storage (audio files)
redis	redis:7-alpine	6379	Celery message broker
backend	Custom (Python 3.12)	8000	Django application server
celery	Same as backend	—	Asynchronous task worker
nginx	Custom (nginx:alpine)	80	Reverse proxy + SPA host

Three named Docker volumes persist data: `postgres_data`, `minio_data`, and `static_files`.

4.2 Nginx Reverse Proxy

Nginx serves four location blocks:

Table 3: Nginx routing rules.

Location	Target
/api/*	Proxy to Django back-end (<code>backend:8000</code>)
/admin/*	Proxy to Django admin interface
/static/*	Serve Django collected static files
/*	Serve React SPA with <code>try_files</code> fallback to <code>index.html</code>

The maximum client body size is set to 50 MB to accommodate audio file uploads.

4.3 Development vs. Production

- **Development:** The back-end uses Django’s built-in `runserver` with `DEBUG=True`; source code is bind-mounted for hot-reload. The Nginx container is disabled (profile `production`); the React dev server runs locally on port 5173 with a Vite proxy for `/api` requests.
- **Production:** Gunicorn serves the Django application behind Nginx. The React SPA is built at image-build time and embedded into the Nginx container. Security hardening is enabled (`HSTS`, secure cookies, `X-Frame-Options: DENY`).

4.4 Build Pipeline

Both the back-end and front-end Dockerfiles use multi-stage builds:

1. **Back-end:** A `python:3.12-slim` builder installs system dependencies and Python packages; the runner stage copies only the installed packages and application code, runs as a non-root `django` user.
2. **Front-end / Nginx:** A `node:22-alpine` builder produces the Vite production bundle; the final `nginx:alpine` image copies the built `dist/` directory and the Nginx configuration.

On container start-up, the back-end entrypoint script automatically runs database migrations, loads exercise fixture data, creates a default superuser, and collects static files before handing off to Gunicorn.

5 Back-end Architecture

The Django project follows a split-settings pattern (`config/settings/{base, development, production}.py`) and contains a single application: `patients`.

5.1 Data Model

The domain model consists of three entities. Figure 2 shows the entity-relationship diagram.

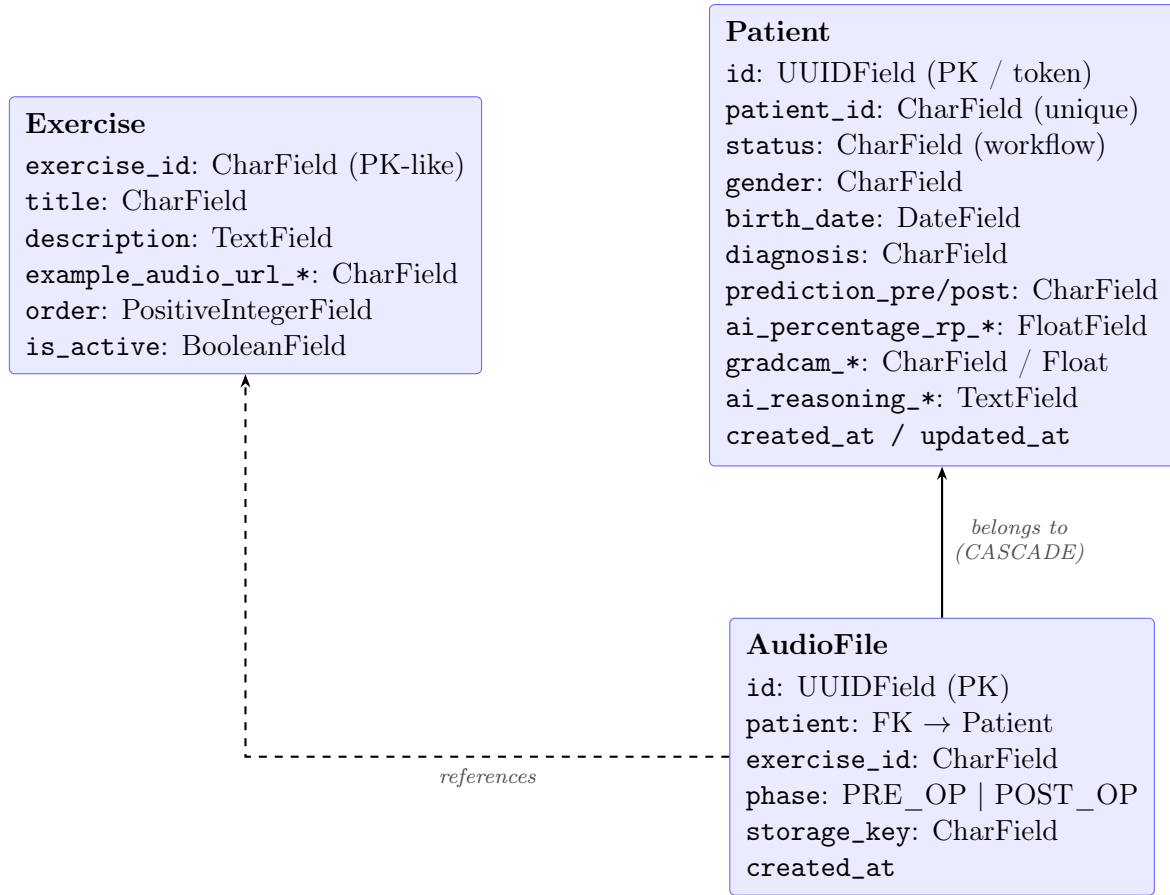


Figure 2: Entity-relationship diagram.

5.1.1 Patient Workflow States

The `Patient.status` field encodes a linear workflow with the following transitions:

NEW → CONSENT_GIVEN → DEMOGRAPHICS_DONE → PRE_OP_DONE → POST_OP_STARTED →
POST_OP_DONE → COMPLETED

Each transition is enforced by the `advance_patient_step()` service function, which also records timestamps (`pre_op_date`, `post_op_date`) and triggers asynchronous inference tasks at the appropriate stages.

5.1.2 Enumerations

Table 4: Model choice fields.

Field	Values
Patient.status	NEW, CONSENT_GIVEN, DEMOGRAPHICS_DONE, PRE_OP_DONE, POST_OP_STARTED, POST_OP_DONE, COMPLETED
Patient.gender	M (male), W (female), D (diverse), ? (unknown)
Patient.diagnosis	LEFT, RIGHT, BOTH, HEALTHY, TODO
Patient.prediction_*	TODO, INFECTED, HEALTHY
AudioFile.phase	PRE_OP, POST_OP

5.2 REST API

The API is built with Django REST Framework and split into three groups based on authentication requirements.

5.2.1 Authentication Endpoints

Table 5: JWT authentication endpoints.

Method	URL	Description
POST	/api/auth/token/	Obtain JWT access + refresh token
POST	/api/auth/token/refresh/	Refresh an expired access token

Access tokens expire after 2 hours; refresh tokens after 7 days and are rotated and black-listed upon use.

5.2.2 Admin Endpoints (JWT Required)

Table 6: Admin API endpoints.

Method	URL	Description
GET	/api/patients/	List all patients (paginated)
POST	/api/patients/	Create new patient
GET	/api/patients/{id}/	Retrieve patient details
PATCH	/api/patients/{id}/	Update patient fields
DELETE	/api/patients/{id}/	Delete patient and S3 files
POST	/api/patients/{id}/advance/	Advance workflow step
GET	/api/patients/{id}/completeness/	Check data completeness
GET	/api/patients/{id}/pdf/	Download QR-code PDF
GET	/api/export/	Export CSV + audio ZIP

5.2.3 Patient-Facing Endpoints (UUID Token)

These endpoints use the patient's UUID primary key as an access token embedded in QR codes, requiring no separate authentication.

Table 7: Patient-facing API endpoints.

Method	URL	Description
GET	/api/p/{token}/	Get public patient data
PATCH	/api/p/{token}/	Update demographics
POST	/api/p/{token}/advance/	Advance workflow step
POST	/api/p/{token}/audio/upload/	Upload audio file (server-side)
POST	/api/p/{token}/audio/presign/	Obtain pre-signed upload URL
POST	/api/p/{token}/audio/confirm/	Confirm pre-signed upload

5.2.4 Public Endpoints

Table 8: Public API endpoints.

Method	URL	Description
GET	/api/exercises/	List active voice exercises
GET	/api/audio/{id}/	Stream audio file from S3
GET	/api/audio/{id}/url/	Pre-signed download URL (JWT)

5.3 Serialisers

DRF serialisers control field exposure and validation:

Table 9: DRF serialiser classes.

Serialiser	Purpose
PatientListSerializer	Compact list view with computed audio_count_pre/post
PatientDetailSerializer	Full detail with nested audio files, age computation
PatientPublicSerializer	Restricted view omitting AI and sensitive fields
PatientCreateSerializer	Accepts only patient_id with uniqueness validation
PatientUpdateSerializer	Demographics, diagnosis, and AI result fields
ExerciseSerializer	Read-only exercise metadata
AudioFileSerializer	Full audio file metadata (admin)
AudioFileCompactSerializer	Compact audio metadata
CompletenessSerializer	Boolean complete, lists of missing/warnings

5.4 Permissions

Table 10: Custom DRF permission classes.

Class	Logic
<code>IsAdminUser</code>	Requires JWT-authenticated user
<code>IsPatientTokenValid</code>	Validates UUID token in URL maps to existing patient
<code>IsAdminOrPatientToken</code>	Allows access with either JWT or valid patient token

5.5 Business Logic Services

Domain logic is centralised in `patients/services.py`:

Table 11: Service layer functions.

Function	Description
<code>advance_patient_step()</code>	State machine enforcing valid workflow transitions; sets timestamps and triggers inference tasks
<code>check_completeness()</code>	Validates that all required demographics, diagnosis, and audio files are present
<code>generate_patient_pdf()</code>	Produces a PDF with a QR code linking to the patient wizard
<code>export_patients_zip()</code>	Generates a ZIP archive containing a CSV summary and all audio files for completed patients
<code>delete_patient_with_files()</code>	Deletes S3 objects before cascading the database delete
<code>generate_presigned_upload_url()</code>	Creates a pre-signed S3 PUT URL for direct client-to-storage uploads
<code>generate_presigned_download_url()</code>	Creates a pre-signed S3 GET URL
<code>upload_audio_to_s3()</code>	Server-side upload of audio data to S3
<code>get_audio_from_s3()</code>	Downloads audio bytes from S3

5.6 Asynchronous Tasks

Celery is configured with a Redis broker and `django-db` result back-end. A single shared task is defined:

`run_inference_task(patient_id, phase)` Calls the external inference service at `/predict` and `/reasoning` endpoints. The payload includes the S3 bucket, audio file keys, patient gender, and age. Results (prediction label, confidence percentage, Grad-CAM outputs, reasoning text) are stored back on the `Patient` model. The task retries up to 3 times with a 30-second delay.

5.7 Dependencies

Key Python packages (from `requirements.txt`):

- Django ≥ 5.1
- djangoestframework ≥ 3.15
- djangoestframework-simplejwt ≥ 5.3
- django-cors-headers ≥ 4.4
- django-filter ≥ 24.0
- django-storages[s3] ≥ 1.14
- boto3 ≥ 1.35
- celery[redis] ≥ 5.4
- django-celery-results ≥ 2.5
- gunicorn ≥ 22.0
- psycpg2-binary ≥ 2.9
- reportlab ≥ 4.2
- qrcode[pil] ≥ 8.0
- requests ≥ 2.32

6 Front-end Architecture

The front-end is a React 19 single-page application scaffolded with Vite 8 and written in TypeScript. It uses Material UI (MUI) 7 for the component library and Axios for HTTP communication.

6.1 Routing

Table 12: Client-side routes.

Path	Component	Auth	Description
/login	LoginPage	Public	Admin login form
/	DashboardPage	Protected	Patient list, create, export
/details/:token	PatientDetailsPage	Protected	Full patient detail view
/p/:token	PatientWizardPage	Public	Patient-facing data collection wizard

Protected routes check for a valid JWT in `localStorage` and redirect to `/login` if absent.

6.2 API Client

Two Axios instances are configured in `src/api/client.ts`:

- **api** — Admin requests with an interceptor that attaches the JWT Bearer token and automatically refreshes expired access tokens via the refresh endpoint.
- **publicApi** — Unauthenticated requests for the patient-facing wizard flow.

Token lifecycle functions (`setTokens`, `clearTokens`, `isLoggedIn`) use `localStorage` keys `access_token` and `refresh_token`.

6.3 Patient Wizard Flow

The patient-facing wizard renders different screen components based on the current `status` value:

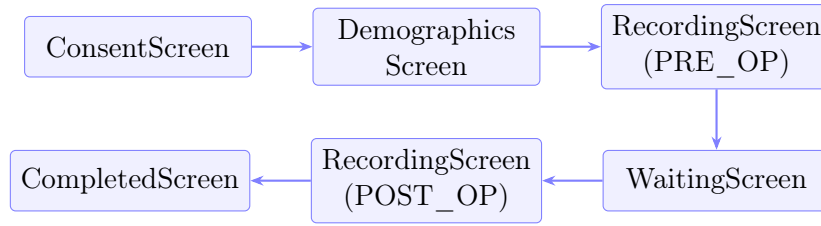


Figure 3: Patient wizard screen flow.

Table 13: Wizard screen–status mapping.

Patient Status	Screen Component
NEW	ConsentScreen
CONSENT_GIVEN	DemographicsScreen
DEMOGRAPHICS_DONE	RecordingScreen (phase = PRE_OP)
PRE_OP_DONE	WaitingScreen
POST_OP_STARTED	RecordingScreen (phase = POST_OP)
POST_OP_DONE / COMPLETED	CompletedScreen

6.4 Audio Recording

The `AudioRecorder` component uses the browser `MediaRecorder` API to capture voice samples. Key features include:

- Real-time waveform visualisation via the `AudioVisualizer` component.
- Recording timer display.
- Playback of the recorded sample before submission.
- Example audio playback (male/female reference recordings).
- Audio quality validation: RMS level is computed in dBFS via the Web Audio API (`AudioContext`), with acceptance thresholds between -30 dBFS and -6 dBFS.

The `useExerciseSession` hook orchestrates the recording session: it loads the exercise list from the API, manages the current exercise index, and handles sequential upload and workflow advancement.

6.5 Component Overview

Table 14: Key front-end components.

Component	Description
<code>PatientList</code>	Sortable, filterable MUI data table with search, status / diagnosis / prediction filters
<code>CreatePatientDialog</code>	Modal dialog to create a new patient record
<code>PatientAccessDialog</code>	Displays the patient access link and QR code
<code>AudioRecorder</code>	Audio capture with waveform visualisation and quality checks
<code>AudioVisualizer</code>	Real-time waveform rendering
<code>ConfirmDialog</code>	Generic confirmation dialog
<code>StatusBadge</code>	MUI Chip displaying workflow status with colour coding
<code>DiagnosisBadge</code>	MUI Chip for diagnosis values
<code>PredictionBadge</code>	MUI Chip for AI prediction results

6.6 Theme

The application uses a custom MUI theme with primary colour `#1565c0`, secondary colour `#f57c00`, and a light background (`#f0f4f8`) using the Roboto typeface.

7 Audio Upload Flow

Two upload strategies are supported to accommodate different deployment scenarios:

1. **Server-side upload:** The audio blob is sent as `multipart/form-data` to the Django back-end, which writes it to S3/MinIO. This is simpler but routes all data through the application server.
2. **Pre-signed URL upload:** The client requests a pre-signed S3 PUT URL from the back-end, uploads the file directly to MinIO/S3, and then confirms the upload. This bypasses Django for the actual file transfer, reducing server load.

In both cases, an `AudioFile` database record is created linking the patient, exercise, phase, and S3 storage key.

8 AI Inference Integration

After the pre-operative (or post-operative) recording phase is completed, the `advance_patient_step()` function dispatches a Celery task (`run_inference_task`) that:

1. Collects the S3 keys for all audio files of the relevant phase.
2. Sends a prediction request to the external inference service at `{INFERENCE_SERVICE_URL}/predict` with the audio storage keys, patient gender, and age.
3. Sends a reasoning request to `/reasoning` for explainability outputs.
4. Stores the results on the `Patient` model: prediction label (`INFECTED/HEALTHY`), confidence percentage, Grad-CAM prediction and percentage, and reasoning text.

The task is configured with automatic retries (up to 3 attempts, 30-second delay) to handle transient inference service failures.

9 Data Export

The export endpoint (GET /api/export/) generates a ZIP archive containing:

- **export.csv:** A CSV file with all completed patient records including demographics, diagnosis, AI prediction results, and timestamps.
- **Audio files:** All associated audio recordings downloaded from S3, organised by patient and phase.

This supports offline analysis and integration with external research tools.

10 Security Considerations

- **Authentication:** Admin routes require JWT tokens (access + refresh). Patient-facing routes are secured by unguessable UUID tokens embedded in QR codes.
- **CORS:** Allowed origins are configurable via environment variables; in production, only the application domain is permitted.
- **Production hardening:** HSTS is enabled, cookies are marked secure and HTTP-only, X-Frame-Options is set to DENY.
- **SQL injection:** Mitigated by Django ORM parameterised queries.
- **CSRF/XSS:** Django’s built-in middleware active; DRF uses session-safe authentication classes.
- **File uploads:** Maximum body size enforced at the Nginx level (50 MB); audio files are stored in isolated S3 buckets, not on the application filesystem.
- **Non-root containers:** The back-end runs as a dedicated `django` user inside the container.

11 Summary

The Stimmbandläsion web tool provides a complete clinical data-collection pipeline: clinicians create patient records via an authenticated dashboard, patients complete a guided wizard to provide demographic information and voice recordings, an AI inference service analyses the recordings asynchronously, and all data can be exported for research purposes. The fully containerised architecture ensures reproducible deployments across development and production environments.